Project 3: Minesweeper

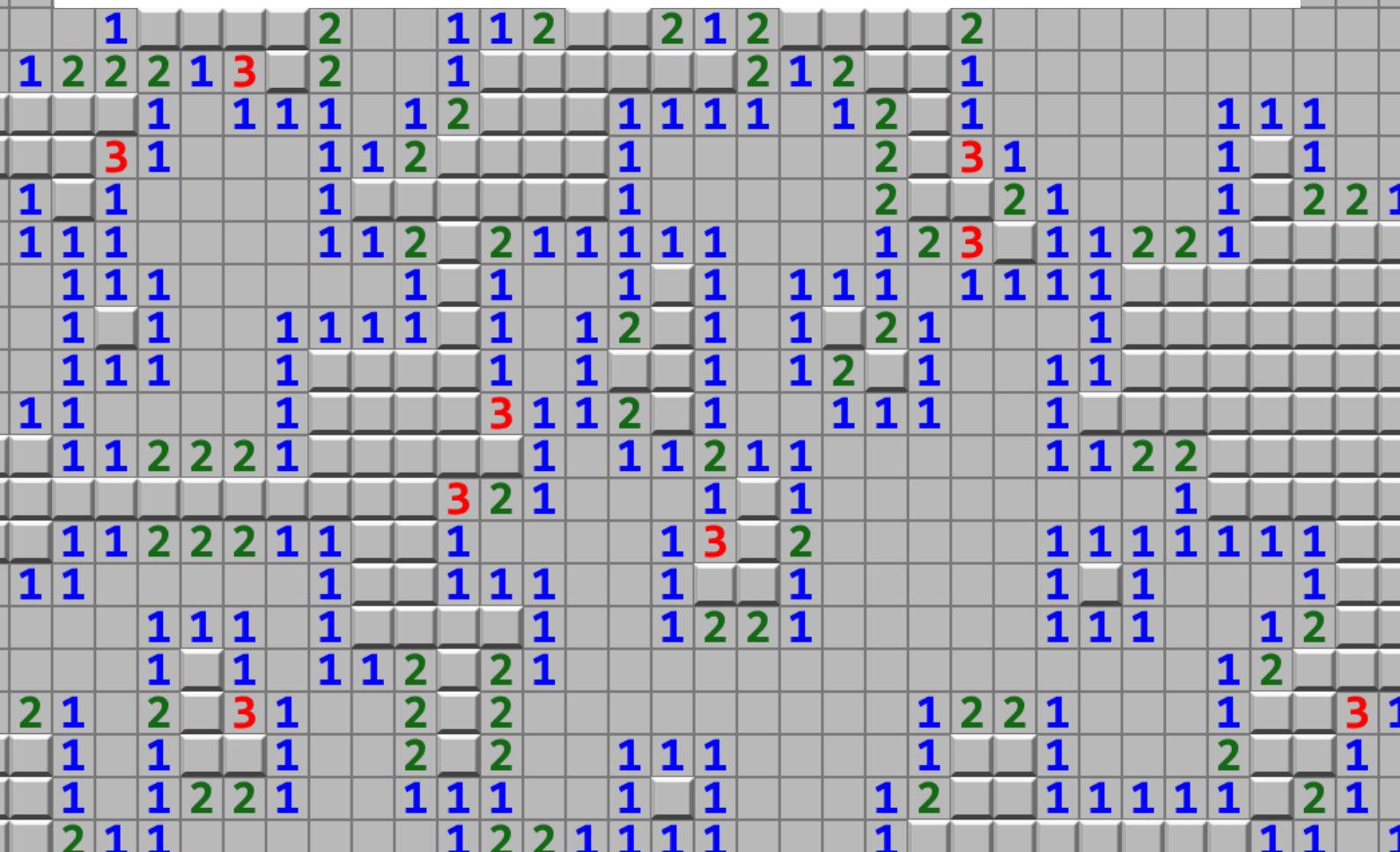
Specifications and Grading

COP3503: Project Fundamentals II

University of Florida

Laura Cruz Castro, Chaitanya Nulu, Cameron Brown,

Joshua Fox,Nathan Bailey



Contents

1 Overview..... 3
2 Milestone Directions..... 3
2.1 Milestone 1 (5 points)..... 3
2.2 Milestone 2 (15 points)..... 3
2.3 Milestone 3 (130 points)..... 5
3 Provided Files..... 6
3.1 config.cfg..... 6
3.2 leaderboard.txt..... 7
3.3 font.ttf..... 7
3.4 images/*.png (Image Files)..... 8
4 Welcome Window Specifications..... 9
4.1 Outline..... 9
4.2 Layout..... 9
4.3 Features..... 10
5 Game Window Specifications..... 11
5.1 Outline..... 11
5.2 Layout..... 11
5.3 Board Generation..... 12
5.4 UI Features..... 12
5.5 Victory..... 13
5.6 Defeat..... 13
5.7 Tile Behavior..... 13
6 Leaderboard Window Specifications..... 15
6.1 Outline..... 15
6.2 Layout..... 16
6.3 Features..... 16

7 Submission..... 17
7.1 Things to Avoid..... 17
7.2 Paths..... 18
8 Grading..... 18
8.1 Rubric..... 19
8.2 Disputes..... 19

1 Overview

Welcome to the third and final project of the course: Minesweeper. Minesweeper is a classic game, around since the 1960s. In this project, you will recreate the classic game *Minesweeper* from scratch using only the c++ standard library and SFML. If you have not played Minesweeper before and are unfamiliar with its rules, please see the provided *About Minesweeper.pdf*.

To recreate Minesweeper, you will use the Simple and Fast Multimedia Library - SFML. SFML handles low-level interactions between hardware and your code. For example, it allows you to draw images to the screen and get input from the mouse. For exactly how to use SFML, please refer to SFML_and_ and the SFML documentation page and the provided *SFML and Coding Tips.pdf*.

NOTICE

Note that, in addition to your own code, you may ONLY use SFML, the C++ standard library, and any code provided through class resources (including lecture slides, Peer Mentor help, lab assignments, etc.). Code segments beyond just a few lines long pulled from other resources (including other former/current students, forums like Stackoverflow, or from generative AI) WILL be considered plagiarism and will be dealt with accordingly. If you are unsure if the use of something counts as plagiarism, please consult a TA.

2 Milestone Directions

The project has three milestones, all graded manually by your peer mentor. For Milestone 1 and Milestone 2, you may either show your peer mentor evidence of completion or submit evidence on Canvas (preferred). Milestone 3 will be submitted on Gradescope.

NOTICE

All milestones must be submitted on time. You will lose all points for any milestone which you do not submit on time. This project takes *at least* many tens of hours to complete, so please, plan accordingly and start as soon as possible.

2.1 Milestone 1 (5 points)

This milestone simply requires following the directions to install SFML and set up an SFML project. For guidance, refer to the directions for Milestone 1 in Canvas.

2.2 Milestone 2 (15 points)

For this milestone, you implement some of the features required in your final submission. Acceptable sets of features are listed in the table below. Other combinations of features might be considered for this milestone; however, as a general, at least a 10-point and a 15-point item of the Milestone 3 rubric will be required for you to achieve full points in Milestone 2.

Record a video demonstrating that all required features work correctly. Upload the video to YouTube or Google Drive, then submit the shareable link on Canvas. Before submitting, verify that the video is accessible in an incognito/private browser window and does not require permission to view.

2.2 Milestone 2 (15 points)

Window	Element	Necessary Implementation
Welcome Window	Player Name	• Player can enter a name up to 10 characters (alphabet only, no numbers/special characters). • Characters appear as typed; backspace removes the last character. • Cursor is visible. • Names are formatted correctly. • Pressing "enter" closes the welcome window and launches the game window. • Closing the welcome window prevents the game window from opening. • The "enter" key has no effect if no characters are entered.
Game Window	Board	• All tiles on the board are visually discernible. • Tile number and dimensions change according to the configuration file. • At least one tile shows correct functionality: 1. Clicking a number tile reveals the number. 2. Right-clicking a tile sets a flag. 3. Right-clicking a flagged tile removes the flag.
Game Window	Mines	• Tiles can be visually seen and revealed. • Mines are randomly assigned to tiles. • At least two buttons (debug and face) are implemented, including the restart game function.
Game Window	Counter	• All tiles are visible, with numbers and dimensions based on the configuration file. • The number of mines is displayed on the counter. • The counter updates to reflect the increase or decrease in mines.
Leaderboard Window	Leaderboard	• The leaderboard reads from *leaderboard.txt* and displays its contents. • Closing the leaderboard window returns to the game window with the timer displayed. • The functionality of the timer is not required at this point.

Table 2.1: Summary of Features to be Implemented for Milestone 2

2.3 Milestone 3 (130 points)

This milestone will be your final, complete implementation of Minesweeper. You will submit a zip folder to Gradescope, containing the following (and nothing else): all .cpp and .h files for your project code, and a readme file. This project is manually graded, and you must ensure that your project complies with all rubric requirements before submitting your folder. Table 2.2 outlines what features must be implemented and how many points they are worth. Note that section 4, section 5, and section 6 contain more detail about exactly how each feature should function.

Item	Points	Description
config.cfg	10	Board changes size and its number of mines based on values set in config.cfg file.
Tile Revealing	15	Clicking on a tile reveals it. If it's a mine, game over. If it has 0 adjacent mines, reveal all neighboring tiles which are not currently revealed, not mines, and not flagged, and then each of those neighbors goes through this process as well. A single click could reveal nearly the entire board depending on the layout.
Tile Display	10	Tiles display depending on their state: • Unrevealed (default state). • Revealed and empty (no adjacent mines). • Revealed, near 1-8 mines (showing the appropriate number). • Revealed, showing a mine.
Flags	10	Right-clicking a hidden tile sets a flag. Right-clicking a flagged tile removes the flag. Left-clicking a flagged tile has no effect. Flagged tiles cannot be revealed unless the flag is removed first.
Mines Remaining	10	A counter shows how many mines are on the board minus the number of flags placed. Flags added or removed affect this count. The remaining flags can go negative.
Victory	10	Revealing all non-mine tiles ends the game. Mines are automatically flagged. Smiley face changes to sunglasses version. No further interactions with the board are possible. The player can start a new game or check the leaderboard. A player's winning time is compared to the leaderboard, and new records are indicated with a '*'.
Defeat	10	Clicking a mine ends the game. All mines are revealed. The smiley face changes to a dead face. No further game board interactions are possible. The player can start a new game or check the leaderboard. Pause-play buttons become inactive.

Table 2.2: Detailed Rubric for Milestone 3 Part I

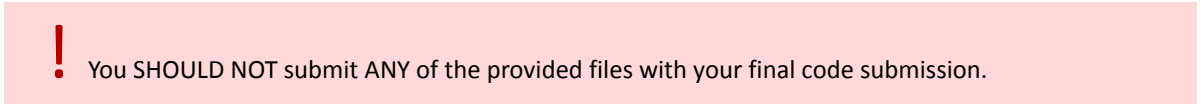
Feature	Points	Description
Random Mine	10	At game start and when resetting (via smiley face button), the specified number of mines from the .cfg file are randomly placed on the map.
Welcome Window	15	Player name entry requirements include: 1. Limit to 10 characters (alphabet only, no numbers/special characters). 2. Visible characters as typed; backspace removes the last character. 3. Visible cursor. 4. Names converted to the right format when typing (not just when stored). 5. Pressing "Enter" closes the window and launches the game. 6. Closing the welcome window prevents game window opening. 7. "Enter" key has no effect if no characters entered.
Timer and Pause Button	15	The timer functions correctly. Clicking the pause button pauses the counter and shows all tiles with tile_revealed.png. Only leaderboard and face buttons work in pause mode. Clicking play resumes the game, reverts tiles to prior states, and continues the counter.
Leaderboard and Button	15	Clicking the leaderboard button shows the top 5 leaders. This action pauses the timer and changes all tiles to tile_revealed.png. Closing this window returns to the game with tiles in their prior states (including the paused mode).
Total	130	

Table 2.3: Detailed Rubric for Milestone 3 Part II

3 Provided Files

The files provided for this assignment can be found on Canvas. You will need the provided files to test your code properly. If you use Clion as recommended, the files directory should be placed *inside* the cmake-build-debug directory. Thus, your files should be within

```
<your_project_directory>/cmake-build-debug/files/*
```



3.1 config.cfg

To correctly size the Welcome Window, the Game Window, and the Leaderboard Window, and to correctly generate the board of the Game Window, you will need to read in data from the text file files/config.cfg. The file contains (in order) the number of columns of the game board, the number of rows of the game board, and

the number of mines, written as plain text on separate lines. Here is an example (and the default), config.cfg file:

```
25
16
50
```

Listing 3.1: Example config file.

Additionally, given *colCount* from the first line of config.cfg and *rowCount* from the second line, here are the formulas for computing the dimensions of the Welcome, Game, and Leaderboard Windows:

Window	Height (px)	Width (px)
Welcome Window	(rowCount ×32)+100	(colCount ×32)
Game Window	(rowCount ×32)+100	(colCount ×32)
Leaderboard Window	(rowCount ×16)+50	(colCount ×16)

Table 3.1: Dimensions of the Welcome Window

Finally, the contents of the file will also abide by these constraints:

- The contents will not be changed while your program is running; only before it is run
- The dimensions will never be less than 22 columns x 16 rows
- The number of mines will be a valid number less than or equal to the number of tiles

We encourage you to modify the config file to help test your code; for example, setting the "mine count" value to 1 in the config file will make it really easy to test victory, and setting it to 399 will make it really easy to test defeat.

3.2 leaderboard.txt

leaderboard.txt contains up to 5 leaderboard entries (and no more). Each entry is on its own line, listing two values, separated by a comma: first, the user's play time, and second, their name. The following is an example (note that the line numbers are not part of the file):

```
1 07:01 ,Alex
2 09:04 ,Bruce
3 10:03 ,Carl
4 11:24 ,Daniel
5 12:15 ,Eve
```

Listing 3.2: The default configuration of leaderboard.txt you are provided with.

You will need to read the *leaderboard.txt* file and, within the Leaderboard Window, display its contents according to the format shown in Figure 6.1. Note that you need to print the ranks on the left-hand side as shown in the figure, but the rankings are not stored in the *leaderboard.txt* file.

Finally, whenever a player wins (in the Game Window), you must check if the current player's win time beats any of the times in *leaderboard.txt*. If it beats one of them, the name of the player and their time should be written to the file in the proper format. Note that the new entry should be *displayed* with an asterisk (*) after the player's name, **but the asterisk should not be written to the file**. Only ever keep the top five player times in **the file**; you must remove the worst entry if there are five entries in the file before writing a new one.

Note that the win-time-check is agnostic of the current game config; a win on a 5x5 board with 1 mine should be checked against wins on a 100x100 board with 1000 mines, regardless of the obvious difficulty difference.

3.3 font.ttf

You will need to render text using sf::Text objects in both the Welcome Window and the Leaderboard Window. sf::Text objects will require sf::Font objects to render the text with the correct font. In turn, the sf::Font objects will require a .ttf, which stores information on how to render the font. You will not need to worry about the inner workings of .ttf files; sf::Font objects handle that for you. For all rendered text in your Minesweeper game, you must use *font.ttf*

3.4 images/*.png (Image Files)

All image files are located within the images directory within the *files* directory. Please ensure that you always reference the image files within the `files/images/ directory` (which should, in turn, be within your `cmake-build-debug/ directory`.

Now, we will take a look at the image files used for the board of the Game Window:

Icon	Filename	Description
	tile_hidden.png	Texture for when tiles are "hidden" (for before they are clicked/revealed)
	tile_revealed.png	A revealed tile with no adjacent mines, or a texture displayed when the game is paused or when the Leaderboard Window is open
	flag.png	Displayed on top of tile_hidden.png when a tile is flagged, either when a player right-clicks a hidden tile or when all mine tiles are flagged automatically when the player wins
	mine.png	Displayed on top of tile_revealed.png when a mine tile is revealed, either when debug mode is active, when a mine tile is clicked, or through being revealed when the player loses.
	number_#.png	Tiles with the numbers 1-8 on them. Displayed on top of tile_revealed.png when a non-mine tile with 1-8 adjacent mines is revealed (either through left-clicking a hidden tile or by being revealed from a neighboring tile). Note the number displayed (and hence the texture chosen) should be the number of mines adjacent to the tile; a tile with 5 adjacent mine tiles should display number_5.png.

Table 3.2: Description of Game Images

Next, let's take a look at all the images you'll need to make the buttons of the Game Window:

Icon	Filename	Description
	face_happy.png	Click this button to reset the map. New mines, everything hidden, it's like you restart the program.)
	face_win.png	Victory!
	face_lose.png	The opposite of Victory! (It's cool, no smiley faces were harmed during the creation of this project)
	debug.png	Used to toggle debug mode
	pause.png	Used to pause the game.
	play.png	Used to play the game when it's paused.
	leaderboard.png	Used to launch the leaderboard window.

Table 3.3: Description of UI Images

Finally, there is a single texture used to display the digits for the flag counter and the minutes/seconds of the game timer:

Icon	Filename	Description
	digits.png	Used for the digits on the “remaining mines” display and the "timer". You can use this one texture for all the numbers, and change the “Texture Rect” of a sprite to draw a different portion of the image. The size of each digit (and the size of the "Texture Rect" you should use) is 21 x 32 pixels, and each digit would be offset by 21 (the width) times the digit you wanted. You can do this by using sf::IntRect. See https://www.sfml-dev.org/tutorials/2.5/ graphics-sprite.php for more information.

Table 3.4: Description of UI Images

4 Welcome Window Specifications

4.1 Outline

The Welcome Window will be the first window to appear, immediately after the program starts. It will prompt the user for their name, allowing them to type their name with a keyboard.



Figure 4.1: Example of the welcome window.

4.2 Layout

The Welcome window will have the same width and height as the Game Window. The dimensions of the Welcome Window are changed based on the information in board_config.cfg, where *colCount* is the value on the first line (25 by default) and *rowCount* is the value on the second line (16 by default). Note that after creating the window, the window should not be re-sizable. The formulas for the Welcome Window dimensions are as follows:

Window	Height (px)	Width (px)
Welcome Window	(rowCount ×32)+100	(colCount ×32)

Table 4.1: Dimensions of the Welcome Window

Here are properties and positions of the text shown on-screen in the Welcome Window:

4.3 Features 4 WELCOME WINDOW SPECIFICATIONS

Text Object	Bold	Underlined	Color	Size	Position
"WELCOME TO MINESWEEPER!"	Yes	Yes	White	24 px	(width / 2, height / 2 − 150)
"Enter your name:"	Yes		White	20 px	(width / 2, height / 2 − 75)
User-typed name	Yes		Yellow	18 px	(width / 2, height / 2 − 45)

Table 4.2: Properties of Text Elements in the Welcome Window

Note: All text in the Welcome Window needs to be aligned to the center. By default, sf::Text objects have their origins located in the top left corner. You can use the code below to shift the origin to the center and set the text location to position (x, y).

```
void setText(sf::Text &text, float x, float y){
    sf::FloatRect textRect = text.getLocalBounds();
    text.setOrigin(textRect.left + textRect.width/2.0f,
                   textRect.top + textRect.height/2.0f);
    text.setPosition(sf::Vector2f(x, y));
}
```

Listing 4.1: Example of using the setText function.

4.3 Features

The following list outlines required rules and behaviors for how this window show intake and respond to user input. Each rule will be tested in your final submission, so be sure that your Welcome Window obeys these specifications. Note that *how* to display text and *how* to display input will be outlined in a later section.

- The user cannot enter more than 10 characters while typing their name.
- The user has to see the characters as they type and pressing backspace should remove the last character.
- You should also display a cursor indicator '|' as the user types in their name. This cursor is just to indicate where the text is being typed. You don't have to implement any blinking effect or use arrow keys to navigate between the text as this is not an actual cursor. But you can implement these features if you want to.
- The characters should only contain alphabetical characters (letters A through Z). So when a user types a number/special character, you shouldn't read it.

- No matter how the user enters their name, you should capitalize the first letter and set the other letters to lowercase. This correction should be visible as the user types not just when you are about to store the name. Here are some examples:
  - alex → Alex
  - bRUCe → Bruce
  - Cameron → Cameron
- When the user presses the Enter key, this window should close and the game window should open if at least one character has been input. Otherwise, the Enter key should not do anything.
- If the user manually closes this window (via pressing the X button, for example), the program should stop executing, and the game window should not load.

5 Game Window Specifications

5.1 Outline

This window will implement the minesweeper game as shown in Figure 5.3, based on the rules outlined in *About Minesweeper.pdf*. Behaviors for the tiles, counters, and buttons are outlined here.

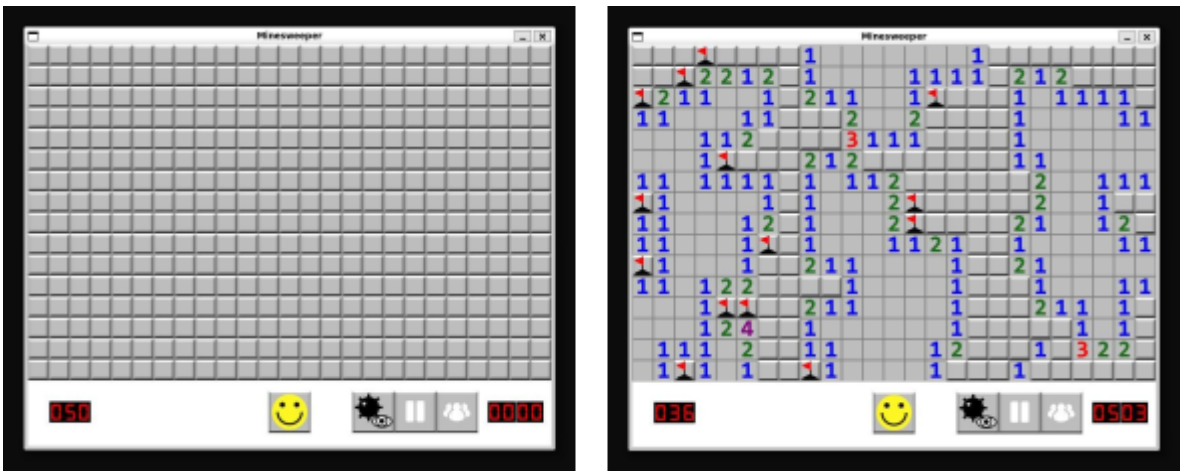


Figure 5.1: Example of the Game Window; Left: The window when the game just starts;
Right: The window during the gameplay

5.2 Layout

The Game Window will have the same width and height as the Welcome Window. The dimensions of the Game Window change based on the information in board_config.cfg, where *colCount* is the value on the first line (25 by default) and *rowCount* is the value on the second line (16 by default). Note that after creating the window, the window should not be re-sizable. The formulas for the Game Window dimensions are as follows:

Window	Height (px)	Width (px)
Game Window	(rowCount ×32)+100	(colCount ×32)

Table 5.1: Dimensions of the Game Window

The positions of each UI element - the buttons and counters - are also based on the column count and row count found in config.cfg. The following table outlines their positions:

5.3 Board Generation

GAME WINDOW SPECIFICATIONS

Component	Position
Happy Face Button	((number of columns / 2.0) × 32) − 32, 32 × (number of rows + 0.5)
Debug Button	(number of columns ×32)−304,32×(number of rows +0.5)
Pause/Play Button	(number of columns ×32)−240,32×(number of rows +0.5)
Leaderboard Button	(number of columns ×32)−176,32×(number of rows +0.5)
Mine Counter	Start drawing the digits from (33,32 × (number of rows +0.5) + 16). For a negative counter, draw the '-' sprite in digits.png at location (12,32 × (number of rows +0.5) + 16). All digits are 21px wide, draw the next digit sprite after 21px.
Timer (Minutes)	Start drawing two digits from ((number of columns ×32)−97,32×(number of rows +0.5)+16).
Timer (Seconds)	Start drawing two digits from ((number of columns ×32)−54,32×(number of rows +0.5)+16). All digits are 21px wide, draw the next digit sprite after 21px.

Table 5.2: Positioning of Various Components

5.3 Board Generation

When the Game Window first opens, and whenever a new game is started (via the face button), you must generate a new, random board. When a new board is generated, it must:

- Have the number of columns and rows specified in config.cfg
- Have the number of mined tiles specified in config.cfg
- Have the mined tiles be evenly randomly distributed across the grid, with the distribution rerandomized every time the board is generated (so they board should look different each time it is reset)
- Begin with every tile being hidden, without any tiles being flagged

Notice

While your board generation algorithm does not need to be incredibly optimized, it should still run at a reasonable speed (less than 15 seconds per board generation) and should not leak memory (so the memory usage of your game should not increase every time the board resets). Poorly-performing or memory-leaking programs will result in deductions.

5.4 UI Features

There are six main UI features, found at the bottom of the Game Window. There are four buttons which should respond with a certain action if they are enabled, then clicked by a player: the Face Button (aka the reset button), the Debug button, the Pause button, and the Leaderboard button. Additionally, there are two counters: The Mine Counter and the Timer. Here is what they do in more detail:

- **Happy Face Button:** This button restarts the game and generates a new, random board. It's sprite changes to face_win.png when the player wins, and to face_lose.png when the player loses.

- **Mine Counter:** Displays the number of mines that are on the board. It begins with the number of mines specified in config.cfg. From there, it decrements when a player flags a tile, and increments when a player un-flags a tile. The counter CAN go negative!
- **Debug Button:** Toggles the visibility of hidden mines on the board ("Debug Mode"). Use this to help you test/debug various features of the game. The debug button shouldn't work when the game ends (victory/loss).

5.5 Victory

- **Pause/Play Button:** Pauses or Plays the game. When the game becomes Paused, the button's sprite should switch to *play.png*, all tiles (regardless of Debug Mode status or current state) should display *tile_revealed.png*, all functionality except the Happy Face button and Leaderboard button should be disabled, and the timer should stop. When the game is Unpaused (by pressing the Play button), all tiles must go back to their previous states (including if they were in Debug Mode or not), all UI functionality should be restored, and the timer should continue from when it was stopped.
- **Leaderboard Button:** When pressed, all tiles (regardless of debug mode status or current state) should display tile_revealed.png sprite, the timer should stop if it is not already paused, the Leaderboard Window should appear, and the player should no longer be able to interact with the Game Window. Note that after the Leader Window closes, the game should be restored to its previous state; e.g., all tiles restored and the timer restarted where it was stopped if the game was unpaused, Debug Mode should be on if it was before, or the game should remain Paused if it was Paused before.
- **Timer:** Displays the amount of time that has passed since the beginning of the game. It should reset to 00:00 whenever a new game starts.

5.5 Victory

Victory is achieved when all non-mine tiles are revealed. Note that this victory condition means that a player could win without having all of the mines flagged (or *any* mines flagged, if you are a Minesweeper speed-runner). Each time a tile or a group of tiles is revealed, you will need to check if victory has been reached. If the victory condition is met, the following should happen:

- The timer should freeze.
- The **Face Button** should change its texture to *face_win.png*.
- Any remaining mine tiles which are *not* flagged should become flagged.
- The mine counter should be set to zero (as a consequence of flagging all of the remaining mines).
- The current time should be checked against the current top-five leaderboard times in *leaderboard.txt*, and the player's name and time should be recorded in *leaderboard.txt* if it beats any of them.

5.6 Defeat

Defeat occurs if the player reveals any mine tiles; they must click on a hidden mine tile which is not flagged. Each time a player clicks on an unflagged, hidden tile, you will need to check if the player has lost. If the defeat condition is met, the following should happen:

- The timer should freeze.
- The **Face Button** should change its texture to *face_lose.png*.
- All remaining mine tiles should be displayed (as if in Debug Mode), regardless of whether or not they are flagged.

5.7 Tile Behavior

Tiles can be in one of several states, and their behavior depends on their state. Additionally, each tile must either contain a mine or not contain a mine. The following table outlines potential states of each tile and how they behave in each state:

5.7 Tile Behavior

State	Base Icon	Top Icon	Behavior
Hidden		*None*	The default state of a tile when the game begins. When right-clicked, the tile becomes **Flagged**. The tile may become **Revealed** either by being revealed by a neighboring tile or by being left-clicked. Note that the rules of **Revealed** tiles are split into multiple cases. Additionally, note that Hidden tiles which contain mines may display the mine texture on top of tile_hidden.png when the game is in Debug Mode.
Flagged			A flagged tile. When a tile becomes Flagged, the mine counter should decrement. A flagged tile cannot be revealed in any way, including when a user left-clicks the flagged tile and when a neighboring tile with zero neighboring mines is revealed and, in turn, attempts to reveal the flagged tile. A user *can* right-click a flagged tile to un-flag it, which reverts the tile to **Hidden** and increments the mine counter.
Revealed contains mine			When a tile which contains a mine becomes Revealed (which should only be possible through left-clicking a Hidden tile), the player loses and the game ends.
Revealed >0 mine neighbors			When a tile with at least one neighboring mine becomes Revealed, the game should check for the victory condition (all tiles which don't contain mines are revealed), and, if the victory condition is met, the player wins and the game ends. Note that the top texture should be *number_#.png*, where # is the number of neighboring tiles which contain a mine.
Revealed =0 mine neighbors		*None*	When this tile becomes Revealed, it should convert every neighboring Hidden tile to Revealed (Note that this does not include neighboring Flagged tiles or neighboring Revealed tiles). Be aware that those tiles, in turn, can also reveal their neighbors if they also neighbor zero mine tiles. Once the neighbor-revealing process finishes, the game should check for the victory condition (all tiles which don't contain mines are revealed), and, if the victory condition is met, the player wins and the game ends.

Table 5.3: Potential states of tiles

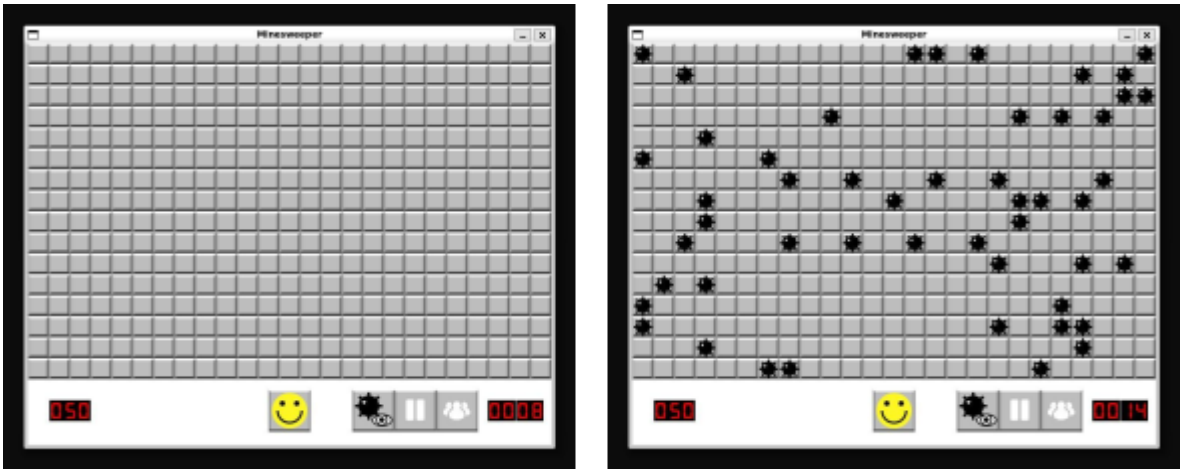


Figure 5.2: Example of the Debug button functionality; Left: Before pressing debug button; Right: After pressing debug button;

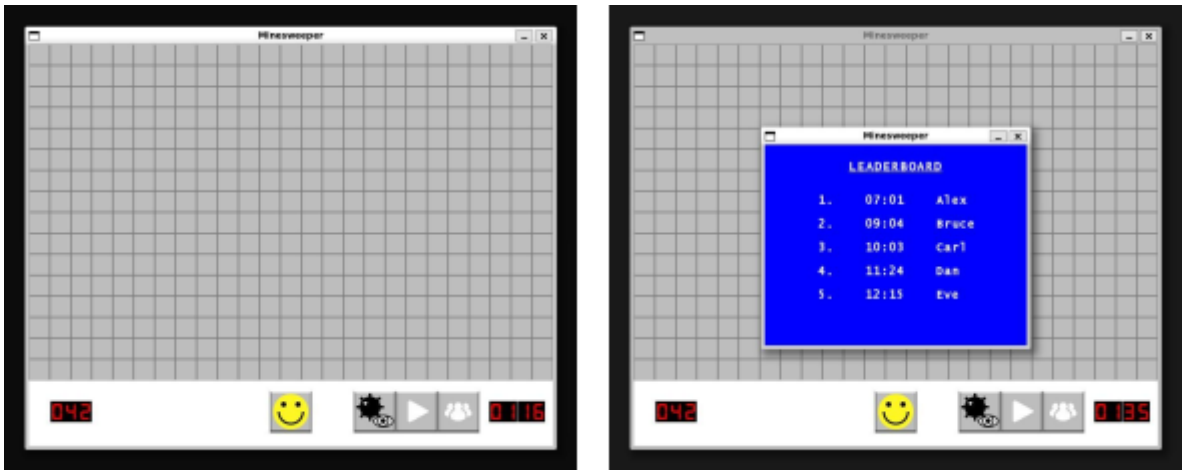


Figure 5.3: Example of the Paused Game Window; Left: The window when the pause button is pressed; Right: The window when the leaderboard button is pressed; (Even when debug mode is active, all tiles should display tile_revealed.png sprite)

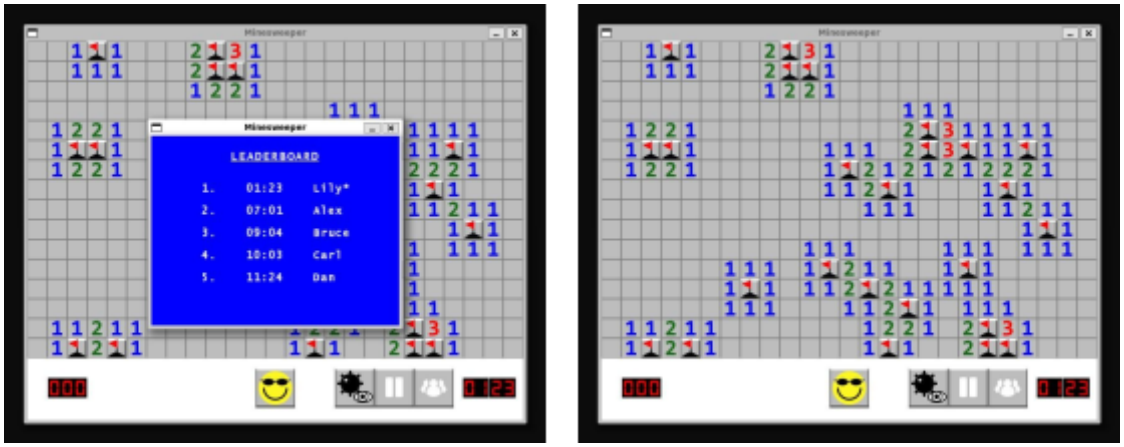


Figure 5.4: Example of the Victory Condition; Left: The leaderboard window pops up when the game is won; Right: The game board after the victory condition is met; Note: This is a 25x16 board and has 25 mines!

6 Leaderboard Window Specifications

6.1 Outline

The Leaderboard Window is a window *separate* from the Game Window appearing when the "Leaderboard" button is clicked within the Game Window. When opened, it displays the top 5 player scores, ranked by time taken to win the game. The score data should be fetched from leaderboard.txt, which lists players' names and their scores in a formatted manner.



Figure 6.1: Example of the Leaderboard Window.
Left: The window when the player clicks the leaderboard button
Right: The window when the player overtakes someone, after winning the game.

6.2 Layout

The Leaderboard Window will have half the height and half the width of Welcome Window and the Game Window. The dimensions of the Leaderboard Window change based on the information in board_config.cfg, where *colCount* is the value on the first line (25 by default) and *rowCount* is the value on the second line (16 by default). Note that after creating the window, the window should not be re-sizable. The formulas for the Leaderboard Window dimensions are as follows:

Window	Height (px)	Width (px)
Leaderboard Window	(rowCount ×16)+50	(colCount ×16)

Table 6.1: Dimensions of the Game Window

When displaying text in the leaderboard window, you should continue to use the same font family provided to you (ie, files/font.tff). Table 6.2 has some more necessary characteristics of the text you will be displaying.

Table 6.2: Properties of Text Elements in the Leaderboard Display

Text Object	Bold	Underlined	Color	Size	Position
"LEADERBOARD"	Yes	Yes	White	20px	(width / 2, height / 2 − 120)
Content of leaderboard.txt	Yes	No	White	18px	(width / 2, height / 2 + 20)

Caption:
Table 6.2: Properties of Text Elements in the Leaderboard Display

Table 6.2: Properties of Text Elements in the Leaderboard Display

It is recommended to store all the rows in a string where each entry is separated by two new line characters (\n\n). Also, you can separate each value in a row with a tab character (\t). Once you have your string, you can proceed with making your *sf::Text* instance. As in earlier sections, remember to use the *setText()* method given to you in Listing 4.1 to ensure the text is center-aligned.

6.3 Features

Your Leaderboard Window should obey the following behavior:

- When a player clicks on the "Leaderboard" button, the Leaderboard Window should open. It should be in the format specified in subsection 6.2 and should display each entry listed in leaderboard.txt.
- The Leaderboard Window should appear *separately* from the Game Window. Note that on Mac devices, it may initially open below the Game Window, rather than above it; this is fine.
- When the Leaderboard Window opens, the Game Window must:
  - Change all tile textures to tile_revealed.png, with no other textures on top, regardless of game status, tile type, etc.
  - Pause the timer, if it is not already paused. Note that if the Game Window is already paused when the Leaderboard Window open, then it should remain paused when the Leaderboard Window closes.
- As long as the Leaderboard Window is open, you cannot interact with the Game Window.
- Whenever a player wins, you check if the current player's win time beats any of the times in leaderboard.txt. If it beats one of them, and their name and time are written to leaderboard.txt, then an asterisk - * -should be displayed after their time/name within the window. *Note that this asterisk should not be written to the file at any point*. If the current player had already set a leaderboard time on a previous game within this same session, that previous record should no longer display with an asterisk; *only* the new time should be displayed with an asterisk.
- When a player closes the Leaderboard Window (by clicking the "X" button of the window), the Game Window should revert exactly to its previous state, including:
  - Reverting all tile sprites to their previous appearance and state - flagged tiles should still display flags, hidden tiles should still be hidden, revealed tile still revealed, etc.
  - Unpausing the timer if the game was unpaused before the Leaderboard Window opened.
  - Remaining paused if the game was paused before the Leaderboard Window opened. – Staying in Debug mode of the the game was in Debug mode previously.

7 Submission

You will turn in your source code and a README.md file only. No images, no SFML libraries, only the .h and .cpp files you wrote to complete the project and a README.md file with the following information: **README.md**

```
Name:
Section:
UFL email:
System:
Compiler:
SFML version:
IDE:
Other notes:
```

Note: Zip up YOUR SOURCE CODE along with this above file filled with all the information.

Folder name: lastname_firstname_project3_Spring2026

Include all fields in the README.md file, if you have nothing to mention in "Other notes", just type "None".

Then submit this zip folder to Canvas (you will be directed to gradescope similar to your experience with Project 2). This project is manually graded, and you must ensure that your project complies with all rubric requirements before submitting your folder.

7.1 Things to Avoid

If the name or your folder does not comply with the naming convention, you will have a -10 point deduction in your project grade.

If you do not use relative file paths, you will have a -10 point deduction in your project grade.

If you include files other than your code and the readme.md, you will have a -10 point deduction in your project grade.

If you do not include the readme.md, you will have a -10 point deduction in your project grade.

If your project uses SFML 2, you will have a -10% reduction in your project grade.

In this project, you should not, under any circumstances, attempt to use sf::Texture and sf::Font objects in global space. Globals in general should be avoided, but if you try to load a sf::Texture/sf::Font in global space it may be initialized before other parts of SFML are initialized, causing it to crash before main() even starts. To make matters worse, this problem might not occur on your machine, but it could be on someone else's (i.e. the person grading your project).

7.2 Paths

Here's the "files" folder structure you will be provided with:

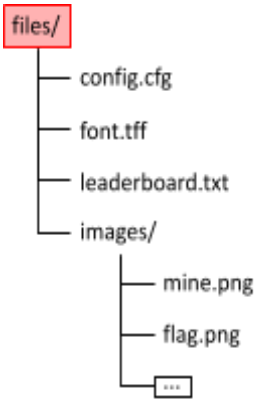


Figure 7.1: The appropriate tree structure for your files.

In this project, any operations involving files (loading textures and boards) should use RELATIVE paths, NOT absolute paths. Your code should be based on the folder structure shown above. When you load a texture, it

should be from the images folder, like “files/images/mine.png”, and when you load the config file, it should be from files/config.cfg.

Be sure to use FORWARD slashes in your paths for compatibility. For example:

- “files\\images\\mine.png” – this will work on Windows, but not elsewhere. Don’t use this.
- “files/images/mine.png” – this will work everywhere. Do use this!

8 Grading

In general, the features you need to implement for this project are listed in the feature summary in the Milestone 3 section 2.2 and 2.3, along with point values. You can get full, half, or no points for each feature.

- Full points: Feature works perfectly, no bugs of any kind
- Half points: Feature has any bugs at all or is partially implemented (if your program is slow, you might get deductions here)
- No points: Feature not implemented at all, or so minimally implemented that no functionality exists (for example: drawing a button to the screen that you can’t click on at all doesn’t count as partial implementation)

8.1 Rubric

The general breakdown of the Project three grade can be found in 8.1

Standard Points		
Item	Points	Description
Milestone 1	5 points	Successful completion of Milestone 1.
Milestone 2	15 points	Successful completion of Milestone 2.
Milestone 3	130 points	Successful completion of Milestone 3.
Total		150
Deductions		
Using Global Variables	-10	Using global variables (including sf::Texture objects).
Not Using Relative Paths	-10	Not using relative paths for file referencing.
Unrequested Files	-10	Existence of files in the submitted folder that were not requested.
Missing README.md	-10	The **README.md** file was not included in the submission.
Using SFML 2	-10% of total	Using a version of SFML before SFML 3, or not using SFML at all.

Table 8.1: Rubric for the Entire Project

8.2 Disputes

This project is manually graded, so we may make mistakes in your grading. After submission, please be sure to regularly check your grades and communications from your TA about their grading progress. If you think there is a mistake with your grade, please contact your lab TA or the TA who graded your submission about it AS SOON AS POSSIBLE. There is only a limited amount of time between when our grading process finishes and when final grades are due, so if you do not attempt to resolve any grading disputes quickly, you may lose your chance.